

NAS 智能存储平台技术架构设计说明书

属性	内容
文档编号	NAS-ARCH-2026-001
版本	V1.1
密级	内部技术文档
编制日期	2026 年 6 月

版本	日期	修改人	修改内容
V1.0	2026-06-21	刁晨钊	初版完成
V1.1	2026-06-22	刁晨钊	修正移动端三级连接回退机制顺序；补充 NFC 登录说明；优化技术架构描述；重构文档全部技术图表，重新绘制系统架构图、功能模块图及流程图，统一图形规范和展示风格

目录

- 项目背景与设计目标
- 总体架构设计
- 技术架构全景
- 技术栈设计
 - 4.1 技术栈总览
 - 4.2 后端技术栈
 - 4.3 身份认证技术栈
 - 4.4 文件服务技术栈
 - 4.5 数据存储技术栈

- 4.6 前端技术栈
 - 4.7 AI 技术栈
 - 4.8 容器与部署技术栈
 - 4.9 安全技术栈
5. 身份认证体系设计
 6. 权限控制体系设计
 7. 文件服务体系设计
 8. 自研核心服务设计
 9. 智能服务体系设计
 10. 移动接入与设备发现设计
 11. 审计日志与可信存证体系
 12. 数据存储架构设计
 13. 网络与容器架构设计
 14. 安全架构设计
 15. 开源组件与自研模块说明
 16. 技术优势与架构特点
 17. 技术演进路线

1 项目背景与设计目标

1.1 建设背景

随着数字化进程加速，数据已成为企业与家庭最核心的生产要素。传统 NAS 产品普遍存在协议孤岛、权限不统一、扩展能力受限、智能化程度低等历史包袱。与此同时，人工智能技术的普及对本地化知识管理、数据安全合规和边缘计算能力提出了更高要求。

本平台正是在上述背景下立项建设——以统一身份、统一权限、多协议融合为基础底座，以智能检索与可信存证为差异化能力，构建面向企业与高价值家庭用户的下一代智能存储平台。

1.2 行业现状

当前市场上主流 NAS 产品存在以下共性问题：

- **权限割裂**：SMB、NFS、HTTP API 各自维护独立的权限体系，变更需多处同步，极易漂移。
- **身份孤岛**：用户账号散落于各子系统，缺乏统一的目录服务，无法支撑跨协议身份联动。
- **智能能力缺失**：仅提供静态文件浏览，无法理解文件内容，无法回答"我的合同文件里有哪些到期提醒"此类语义查询。
- **存证能力薄弱**：操作日志可被静默篡改，无法满足合规审计与法律举证的可信度要求。
- **移动体验差**：依赖手动填写 IP 地址，局域网自动发现与近场无密码接入能力缺失。
- **国产化适配不足**：缺乏国密算法与国产硬件安全模块的扩展路径。

1.3 目标用户与目标市场

目标用户

用户群体	核心诉求
中小企业 IT 部门	统一账号管理、多协议文件共享、合规审计
企业知识管理团队	文件语义检索、AI 知识库、版本追溯
教育机构	多用户隔离、共享资源权限管控、低运维成本
安防与视频业务	大容量写入、目录结构权限、审计留痕
高价值家庭用户	移动端零配置接入、照片自动备份、NFC 便捷登录

目标市场

本平台定位于个人/家庭高端市场与企业轻量化 NAS 市场的交叉地带，重点覆盖以下场景：

- **企业办公**：替代 Dropbox/Box 等云盘，数据留存本地，权限精细管控
- **AI 知识库**：将企业文档转化为可检索的知识资产，接入大语言模型
- **教育场景**：多学生账号隔离，教师共享资料，支持 WebDAV 挂载
- **视频监控**：接受大量摄像头写入流，本地存储，操作留痕
- **家庭数据中心**：照片、视频、文档统一管理，手机端随时访问

1.4 建设目标

本平台建设目标如下：

- 1. **统一身份管理**：以 LDAP 目录服务作为全系统唯一用户信息源，所有协议共享同一套账号体系。
- 2. **统一权限管理**：以 POSIX ACL 作为全系统唯一权限真相，所有访问协议透明复用，消除权限漂移。
- 3. **多协议兼容**：同时支持 SMB（Windows）、NFS（Linux）、WebDAV（跨平台）、HTTP API（移动端与程序化访问），满足异构终端接入需求。
- 4. **智能扩展**：通过 Agent + MCP + RAG 架构，将 NAS 文件系统转化为可语义检索的智能知识库。
- 5. **可信存储**：通过哈希链结构的操作日志与可导出的证明包，为合规审计提供不可篡改的操作记录。
- 6. **云边协同**：支持本地优先运行，未来可接入云端存储扩展与远程访问能力。
- 7. **国产化演进**：预留国密算法（SM2/SM3）与 PUF 硬件安全芯片接入接口，支持国产化演进路径。

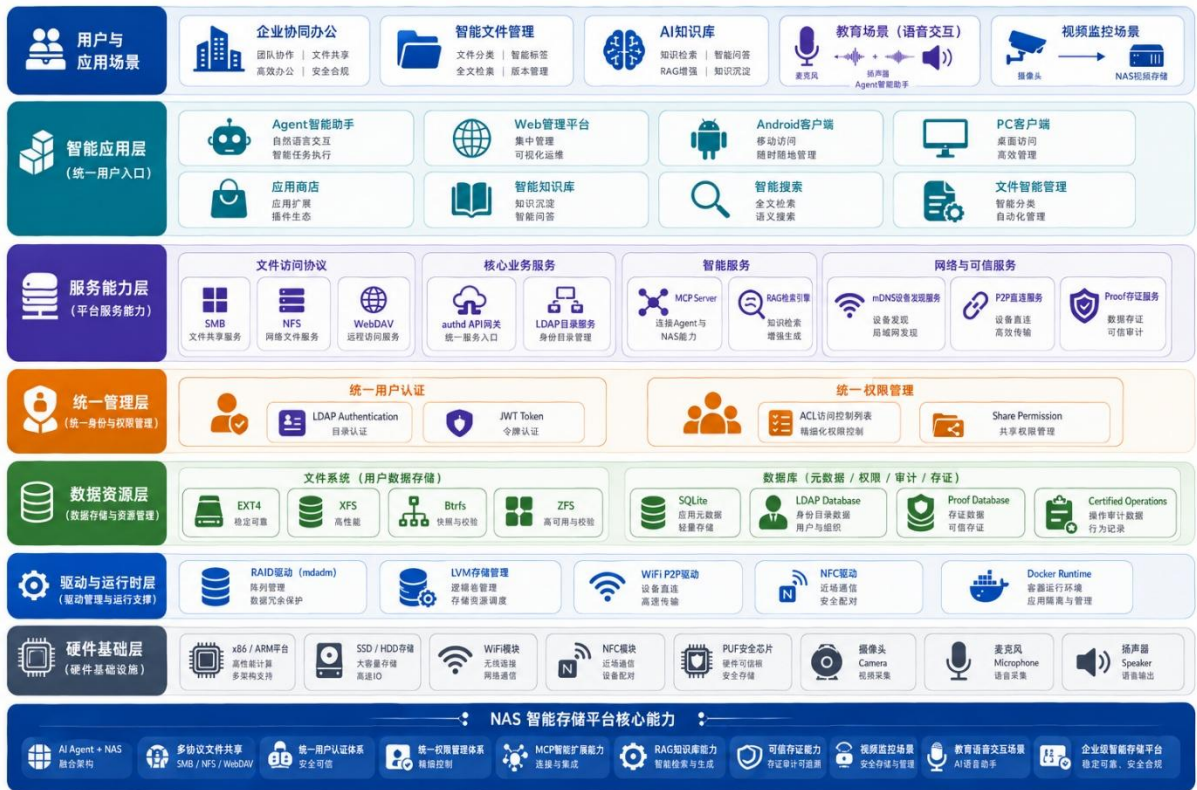
1.5 设计原则

设计原则	具体体现
统一身份管理	OpenLDAP 作为单一身份源，所有协议同库认证
统一权限管理	POSIX ACL 作为单一权限源，跨协议透明复用
多协议兼容	SMB / NFS / WebDAV / HTTP API 并存
智能扩展	Agent + MCP + RAG 架构，可插拔 AI 能力
可信存储	哈希链审计日志，导出可独立验证的证明包
云边协同	本地完整运行，预留云端扩展接口
国产化演进	SM2/SM3/PUF 接口预留，逐步替换国际算法

2 总体架构设计

2.1 架构分层概述

本平台采用七层分层架构，从上至下依次为：用户与应用场景层、智能应用层、服务能力层、统一管理层、数据资源层、驱动层、硬件基础层。各层职责清晰，上下层通过标准接口协议交互，横向扩展不影响相邻层。



2.2 各层职责说明

用户与应用场景层

该层是系统的最终消费者，包括通过 Web 浏览器访问管理后台的运维人员、通过 Android App 接入的移动端用户、通过 SMB/NFS 协议挂载存储的 Windows/Linux 工作站，以及通过 WebDAV 进行文档协作的第三方工具。本层不包含任何业务逻辑，所有访问均通过下方服务能力层提供的标准接口进行。

智能应用层

该层承载平台的差异化智能能力。通过 Agent 接收用户的自然语言指令，借助 MCP（Model Context Protocol）协议将 NAS 文件系统的工具能力暴露给大语言模型，通过 RAG（检索增强生成）流水线在文件内容与元数据中进行语义检索，最终以结构化答案回传用户。智能层不绕过权限体系，所有工具调用须通过统一管理层的权限校验。

服务能力层

该层提供多协议并发服务能力，是不同终端接入 NAS 的统一入口集合。各协议服务均与统一管理层对接，保证身份核验与权限控制的一致性。

统一管理层

该层是整个平台的核心控制面，包含三大子系统：统一身份认证系统（以 OpenLDAP 为目录服务核心）、统一权限控制系统（以 POSIX ACL 为权限执行器）、审计与存证系统（以哈希链结构保证日志完整性）。所有上层服务在执行任何业务操作前，均须先经过本层授权。

数据资源层

该层对数据按用途分类存储：用户身份数据由 LDAP 数据库管理；文件数据存储于 Linux 文件系统；业务元数据（操作日志、哈希链节点等）由 SQLite 数据库管理；存证数据以专用证明包格式持久化，支持离线导出与独立验证。

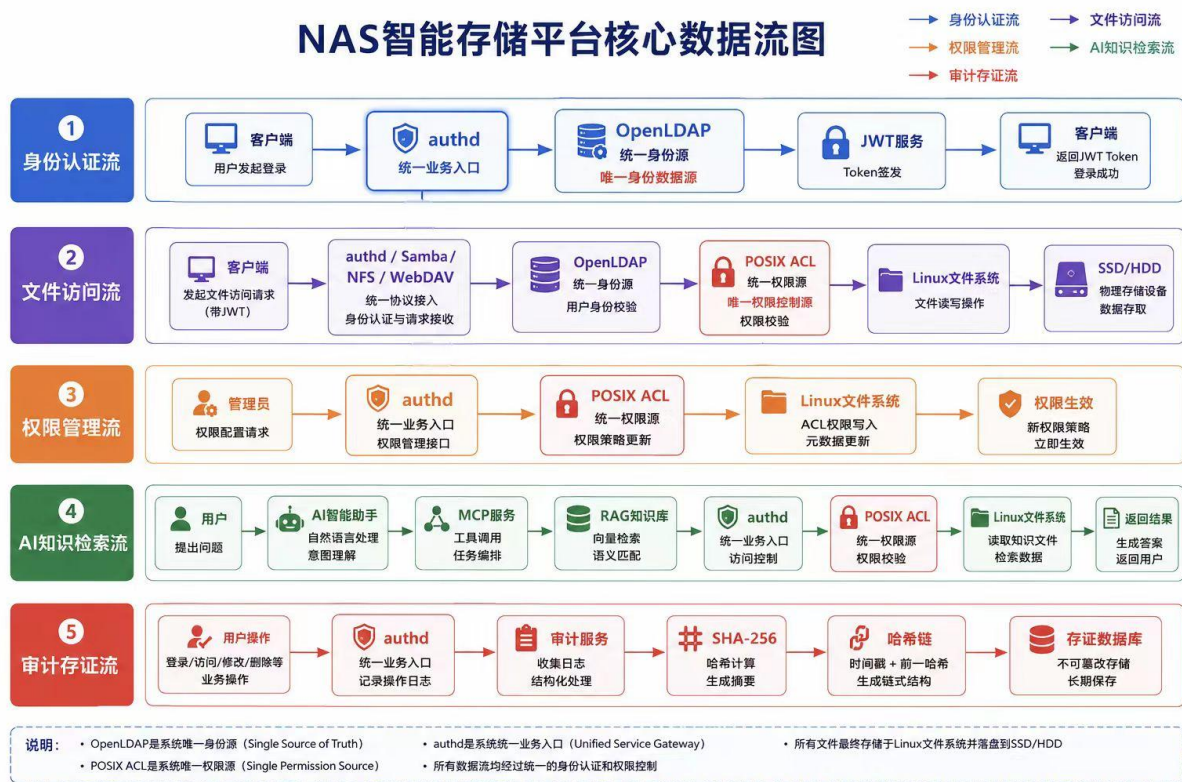
驱动层

该层提供操作系统级别的标准化接口，包括 Linux VFS（虚拟文件系统）、POSIX 系统调用接口、ACL 子系统以及网络协议栈。各上层服务均通过这些标准接口与底层硬件交互，平台移植性因此得到保障。

硬件基础层

该层包括物理存储介质（HDD/SSD/NVMe）、网络接口卡以及规划中的 PUF（物理不可克隆函数）安全芯片。PUF 芯片将作为设备级硬件根信任锚点，为存证签名提供不可伪造的硬件级私钥。

2.3 系统整体数据流



2.4 系统整体运行逻辑

系统启动后，Docker Compose 依次拉起 OpenLDAP 容器（身份数据库）与 NAS 主容器（包含 authd、Samba、NFS、Nginx 四项服务）。OpenLDAP 在主容器启动前完成数据初始化，确保身份源就绪。

主容器内，authd 作为统一业务入口服务监听 8080 端口，接收来自 Web 管理台和移动 App 的 HTTP 请求。Samba 监听 445 端口，对外提供 Windows 文件共享；NFS 监听 2049 端口，对外提供 Linux 文件挂载；Nginx 监听 8081 端口，对外提供 WebDAV 服务。四项服务均以同一组文件系统目录（/data）为数据根路径，通过 Linux 内核的 POSIX ACL 子系统共享同一套权限规则。

mDNS 服务随 authd 启动，向局域网广播设备服务信息，使移动端无需手动配置即可自动发现 NAS 设备。

3 技术架构全景

3.1 核心能力组成

本平台技术架构由七大核心能力子系统构成，各子系统既相互独立、可单独演进，又通过统一管理层形成协同整体。



3.2 身份认证体系

设计目标：构建全系统单一用户信息源，使任何协议的认证请求均查询同一目录，消除多账号同步问题。

核心组成：以 OpenLDAP 作为目录服务核心，扩展 posixAccount、shadowAccount、sambaSamAccount 三套 Schema，覆盖 Linux 系统账号、Unix 密码策略、Windows NT Hash 三类认证需求；自研 authd 服务封装 LDAP 操作，对上层提供统一的注册、认证、用户管理接口；JWT 令牌机制负责 HTTP 层的无状态会话管理。

模块协作：authd 接收注册/登录请求 → 调用 LDAP 客户端完成目录操作 → 签发 JWT 令牌 → 后续 HTTP 请求携带 JWT，authd 中间件解析验证 → 权限层基于 LDAP 用户属性做出授权决策。

3.3 权限管理体系

设计目标：以 POSIX ACL 作为全系统唯一权限执行器，保证 HTTP、SMB、NFS、WebDAV 四种访问路径读到的权限结果完全一致。

核心组成：Linux 内核 ACL 子系统作为权限存储与执行的底层基础；authd 的 permission 模块封装 setfacl/getfacl 系统调用，提供结构化权限管理接口；各协议服务（Samba、NFS、Nginx）直接依赖内核 ACL，不引入额外权限层。

模块协作：用户通过 HTTP API 调用 permission 模块 → authd 校验身份与角色 → 调用系统层写入 ACL → 变更立即对 Samba/NFS/WebDAV 生效，无须重启或同步。

3.4 文件服务体系

设计目标：满足异构终端（Windows、Linux、macOS、移动端、程序化客户端）的文件访问需求，同时保证所有协议的权限一致性。

核心组成：Samba 提供 SMB 协议服务；NFS Server 提供网络文件系统服务；Nginx 提供 WebDAV 服务；authd 提供 HTTP RESTful 文件 API；所有协议共享 /data 目录与 POSIX ACL 权限。

模块协作：四种协议服务并列运行，共享底层文件系统；Nginx WebDAV 通过 auth_request 机制回调 authd 内部接口完成密码验证，保证 WebDAV 的认证也经过统一的 LDAP 身份源。

3.5 智能服务体系

设计目标：将 NAS 文件系统的内容能力暴露给大语言模型，实现自然语言驱动的文件检索与管理。

核心组成：Agent 服务作为自然语言接入层；MCP（Model Context Protocol）协议规范化工具接口；RAG 流水线提供三层混合检索；知识库索引模块维护文件元数据与内容向量。

模块协作：见第 9 章详细说明。

3.6 可信存证体系

设计目标：为每一次文件操作和身份操作生成不可篡改的操作证明，满足合规审计与法律举证需求。

核心组成：certified_operations 表记录操作详情（22 个字段）；proof_records 表以链式哈希结构串联所有操作记录；证明包导出接口生成可独立验证的 JSON 格式存证文件；设备唯一标识作为签名溯源标记。

模块协作：每次文件操作完成后，authd 同步写入 certified_operations；证明链模块计算新的哈希节点并追加到 proof_records；管理员可随时导出完整证明包，第三方持公开验证算法即可独立核验完整性。

3.7 移动接入与设备发现体系

设计目标：实现局域网内零配置自动发现、NFC 近场无密码登录，降低移动端接入门槛。

核心组成：Android 原生应用承载移动端交互；mDNS (Zeroconf) 模块负责局域网服务广播与发现；WiFi P2P 模块提供基础网络不可用时的点对点直连备用通道；NFC 模块实现 NFC 标签绑定与碰一碰免密登录。

模块协作：见第 10 章详细说明。

4 技术栈设计

4.1 技术栈总览

技术领域	技术方案	类别	作用	成熟度
后端语言	Go 1.25	开源	核心业务逻辑开发	生产级
Web 框架	Gin v1.9	开源	HTTP 路由与中间件	生产级
目录服务	OpenLDAP 1.5	开源	统一身份源	生产级
文件共享	Samba 4.x	开源	SMB/CIFS 协议服务	生产级
网络文件系统	NFS (Linux 内核)	开源	NFS 协议服务	生产级
WebDAV	Nginx + DAV 模块	开源	WebDAV 协议服务	生产级
权限模型	POSIX ACL	开源	统一权限执行	生产级
关系数据库	SQLite (modern)	开源	业务元数据与审计数据	生产级
容器运行时	Docker	开源	服务隔离与部署	生产级
容器编排	Docker Compose	开源	多容器生命周期管理	生产级
管理后台框架	Next.js 16 / React 19	开源	Web 管理界面	生产级
前端语言	TypeScript 5.5	开源	类型安全前端开发	生产级
UI 组件库	shadcn/ui + Radix UI	开源	管理台 UI 组件	生产级

技术领域	技术方案	类别	作用	成熟度
移动端框架	React Native 0.85	开源	Android 客户端开发	生产级
移动端原生	Android Kotlin	开源	NFC/WiFi P2P 原生模块	生产级
身份令牌	JWT (HS256)	开源	HTTP 无状态会话	生产级
服务发现	mDNS / Zeroconf	开源	局域网设备自动发现	生产级
设备直连	WiFi P2P	开源	无 AP 直接连接	生产级
近场认证	NFC (HCE/NDEF)	开源标准	碰一碰免密登录	生产级
存证算法	SHA-256	开放标准	哈希链完整性校验	生产级
AI 接入协议	MCP	开源	Agent 工具调用规范	演进中
智能检索	RAG (三层混合)	自研	语义文件检索	演进中
统一业务入口	authd	自研	HTTP API + 权限 + 审计	生产级
管理后台	nas-web	自研	Web 管理界面	生产级
Android 客户端	nas-app	自研	移动端接入	生产级
证明链系统	proof_records 模块	自研	哈希链存证	生产级
管理接口层	admin_* 模块	自研	系统资源统计与用户管理	生产级

4.2 后端技术栈

Go 语言

Go (golang) 是本平台后端唯一的开发语言，版本为 1.25。选择 Go 的核心理由如下：

- **高并发低开销**：Go 原生 goroutine 并发模型支持在单进程内调度数万并发连接，内存占用远低于 Java/Spring 体系，适合 NAS 设备资源受限的运行环境。
- **静态编译单二进制**：Go 可将所有依赖编译进单个可执行文件，无需运行时环境，容器镜像体积小，冷启动快，部署无依赖地狱。
- **跨平台原生支持**：Go 工具链原生支持交叉编译，可在开发机上直接生成 ARM/MIPS 等嵌入式架构的二进制，为后续部署于 ARM SoC 设备的国产化场景奠定基础。

- **标准库完备**：Go 标准库内置 HTTP 服务器、加密算法、系统调用封装等核心能力，第三方依赖引入量少，供应链风险低。
- **类型安全与工程效率**：强类型系统与简洁语法在保证代码健壮性的同时，降低了团队协作的理解成本。

Gin 框架

Gin 作为 Go 生态中性能最优的 HTTP 路由框架之一，提供中间件链（JWT 验证、跨域、请求日志）、路由分组（公开端点/认证端点/管理端点/内部端点）、参数绑定与校验、Swagger 自动文档生成等能力，显著提升了 API 层的开发效率与可维护性。

JWT (JSON Web Token)

authd 采用 JWT HS256 算法签发令牌，有效期 24 小时。JWT 的无状态特性使得服务端无需维护会话存储，天然适合多实例扩展。令牌中携带用户名与角色信息，中间件层可在不查询 LDAP 的情况下完成身份验证，降低了认证热路径的延迟。

LDAP SDK

`go-ldap/ldap/v3` 是 Go 生态中成熟的 LDAP 客户端库，支持连接池、TLS、DN 绑定、条目搜索、属性修改等完整 LDAP v3 操作，用于 authd 与 OpenLDAP 之间的所有目录交互。

SQLite (modernc 纯 Go 实现)

采用 `modernc.org/sqlite`（无 CGO 依赖的纯 Go SQLite 实现），结合 `sqlx` 提供的结构体映射能力，管理审计日志与哈希链数据。纯 Go 实现保证了整个后端二进制仍为静态编译，无外部动态库依赖。

4.3 身份认证技术栈

OpenLDAP

OpenLDAP 是开源 LDAP 目录服务的事实标准实现，以 `/etc/openldap/schema` 定义的 Schema 扩展机制支持多种账号属性体系。本平台采用 `osixia/openldap:1.5.0` 容器镜像部署 OpenLDAP，通过 LDIF 初始化脚本完成目录结构与初始数据的导入。

LDAP Schema 设计

本平台在标准 LDAP 目录中扩展了三套对象类（ObjectClass）：

Schema	对象类	承担职责
posix.schema	posixAccount	存储用户名、UID、GID、主目录路径，支持 NFS UID 映射
posix.schema	posixGroup	存储用户组名与 GID，用于 NFS 组权限
shadow.schema	shadowAccount	存储密码策略（过期时间、最小长度等）
samba.schema	sambaSamAccount	存储 NT Hash，支持 Samba ldapsam 认证后端
自定义	NFC 绑定条目 (ou=nfc_bindings)	存储 NFC 标签 UID 与用户名的映射关系

为什么选择 LDAP 而非关系数据库

传统做法是将用户账号存储于 MySQL/PostgreSQL 等关系数据库，但这种方式存在本质缺陷：Samba 的 ldapsam 认证后端、NFS 的 UID 映射、Linux PAM 模块均原生支持 LDAP，而不支持直接查询关系数据库。若用关系数据库存储用户，则必须在 Samba 与 NFS 之间单独维护账号同步逻辑，引入一致性风险。LDAP 作为目录服务天然成为「统一身份源（Single Source of Truth）」的最优载体：一次变更，所有协议立即生效，无需任何同步。

统一身份源设计思想

所谓统一身份源，是指全系统有且仅有一处存储用户的权威信息，任何子系统在需要用户信息时，均从该唯一来源查询，而非各自维护副本。本平台以 OpenLDAP 实现这一目标：HTTP 认证时，authd 通过 LDAP Bind 验证密码；SMB 认证时，Samba 通过 ldapsam 读取 NT Hash；NFS 认证时，内核通过 libnss-ldap 解析 UID；WebDAV 认证时，Nginx auth_request 回调 authd，authd 再调 LDAP Bind。四条路径殊途同归，都指向同一个 OpenLDAP 目录。

4.4 文件服务技术栈

多协议并存的必要性

企业与家庭环境中终端设备的操作系统高度异构：Windows 工作站原生使用 SMB 协议；Linux 服务器与工作站使用 NFS 进行高性能挂载；macOS 与跨平台工具（如文档编辑器、备份软件）使用 WebDAV；移动 App 与自动化脚本使用 HTTP RESTful API。单一协议无法覆盖全部终端，多协议并存是保障用户体验的必要选择。

Samba (SMB/CIFS 协议)

Samba 是 Linux 上实现 SMB/CIFS 协议的开源套件，是 Windows 文件共享的标准解决方案。本平台配置 Samba 以 `ldapsam` 作为密码认证后端，直接从 OpenLDAP 读取 `sambaNTPassword` 属性进行 NTLM 认证；共享规则中以 `valid users = @nas-users` 限制访问组；底层文件权限由 POSIX ACL 控制，Samba 完全遵从内核 ACL，不引入独立权限逻辑。

NFS (网络文件系统)

NFS 是 Linux/Unix 原生的网络文件系统协议，通过 UID/GID 映射实现权限控制。本平台利用 `libnss-ldap` 使 Linux 系统的用户名解析指向 OpenLDAP，确保每个 LDAP 用户都拥有与 NFS 挂载点一致的 UID，从而透明复用 POSIX ACL 权限，无需额外的 NFS 专属权限配置。

WebDAV (基于 Nginx)

WebDAV 扩展了 HTTP 协议，支持文件的创建、删除、移动、复制、属性查询等操作。本平台采用 Nginx 的 `ngx_http_dav_module` 模块提供 WebDAV 服务，并通过 Nginx 的 `auth_request` 指令将每次请求的身份验证委托给 `authd` 的 `/internal/verify-password` 接口，实现 WebDAV 认证与 LDAP 身份源的联动。

POSIX ACL 的核协作用

POSIX ACL（可访问控制列表）是 Linux 文件系统对传统 Unix 三元权限（Owner/Group/Other）的扩展，支持为任意用户或组设置独立的读/写/执行权限。本平台将 POSIX ACL 作为跨协议的权限统一执行器：无论通过哪种协议访问，Linux 内核在执行文件系统操作时均检查 ACL，因此一次 ACL 变更立刻对所有协议生效，跨协议权限一致性由操作系统内核保证，而非应用层同步。

4.5 数据存储技术栈

本平台采用分层异构存储策略，不同类型的数据由最适合其特征的存储系统管理。

Linux 文件系统 (EXT4 / XFS / Btrfs / ZFS)

用户文件数据存储于 Linux 原生文件系统，挂载于 `/data` 目录。平台不绑定特定文件系统，可根据硬件条件与运维需求灵活选择：

文件系统	适用场景	特点
EXT4	通用场景，默认推荐	稳定成熟，兼容性最好

文件系统	适用场景	特点
XFS	大文件与高吞吐场景	大文件写入性能优异
Btrfs	需要快照与 RAID 功能	写时复制，内置校验
ZFS	企业高可靠场景	端到端数据完整性，内置去重

OpenLDAP 数据库（身份数据）

OpenLDAP 内部使用 LMDB（Lightning Memory-Mapped Database）作为存储后端，这是一个高性能的嵌入式键值数据库，专为 LDAP 目录操作优化。所有用户账号、用户组、NFC 绑定数据均存储于此。LDAP 数据库独立于文件系统存储，通过 Docker Volume 持久化，避免容器生命周期变化导致数据丢失。

SQLite（业务元数据与存证数据）

SQLite 数据库（.nas.db）存储两类关键数据：

- `certified_operations` 表：记录每一次文件操作与身份操作的完整上下文，包含 22 个字段，涵盖操作者、时间戳、路径、文件属性、内容哈希等信息。
- `proof_records` 表：存储哈希链节点，每条记录包含指向前一节点的哈希值，形成不可逆的链式结构。

选择 SQLite 的理由：零依赖、单文件存储、ACID 事务语义完备、无需独立数据库服务进程，完全契合 NAS 设备的轻量化运行需求。采用 `modernc.org/sqlite`（纯 Go 实现）进一步消除了 CGO 依赖，保证后端二进制的静态编译特性。

数据分层设计理念

三类数据（文件数据、身份数据、业务数据）采用完全独立的存储系统，原因在于：各类数据的访问模式、持久化需求、查询特征、安全要求差异显著。文件数据需要高吞吐顺序 I/O；身份数据需要快速树形查询与目录协议支持；业务数据需要关系查询与事务保障。混用同一存储系统既无法满足各自性能要求，也会增加单点故障的影响范围。

4.6 前端技术栈

前后端分离架

本平台 Web 管理平台采用前后端分离架构：Next.js 应用独立构建与部署，通过 HTTP 调用 authd 暴露的 RESTful API，不与后端代码耦合。这一设计的核心收益在于：前后端可独立迭代，前端技术升级不影响后端服务；API 层天然成为平台能力的标准输出，便于第三方客户端接入。

Next.js (App Router 架构)

Next.js 16 采用 App Router 架构，支持服务端渲染 (SSR) 与客户端渲染 (CSR) 混合模式：登录页与首屏可采用 SSR 提升首屏加载速度，文件管理等交互密集页面采用 CSR 保证流畅度。文件系统路由机制 (`/app/files/page.tsx` 对应 `/files` 路径) 降低了路由维护成本。

React 19 与 TypeScript 5.5

React 19 提供了并发渲染能力 (Concurrent Mode)，在文件列表等大数据量场景下通过虚拟列表 (react-virtuoso) 保证滚动流畅性。TypeScript 5.5 的强类型约束覆盖了所有 API 请求与响应的数据结构，在编译阶段即可发现接口不一致问题，大幅降低前后端联调成本。

shadcn/ui 组件体系

shadcn/ui 基于 Radix UI 原语构建，采用「复制到项目」而非「包依赖」的分发模式，组件代码直接进入代码库，无运行时版本依赖，可任意定制。结合 Tailwind CSS 4 的实用类系统，实现了一致的视觉规范与高效的样式开发。

Android Kotlin (原生模块)

移动端 React Native 框架承载大部分跨平台业务逻辑 (网络请求、文件管理、UI 交互)，而 NFC 硬件读写、WiFi P2P 连接管理等需要访问 Android 系统 API 的功能，则通过 Kotlin 编写的原生模块 (Java Script Bridge, JSB) 实现。JSB 模式保证了 JavaScript 层与原生能力的解耦，原生模块升级不影响 JS 业务逻辑。

4.7 AI 技术栈

为什么不让大模型直接读取文件

朴素的方案是将 NAS 中所有文件内容直接喂给大语言模型，但这存在三个根本性问题：其一，模型上下文窗口有限，文件总量远超任何单次请求的处理能力；其二，模型推理成本与上下文长度成正比，全量文件输入经济上不可持续；其三，实时索引所有文件变更对本地硬件压力极大。RAG 架构解决了这三个问题。

RAG (检索增强生成) 架构

RAG 将「检索」与「生成」分离：先通过高效的向量检索在知识库中找出与用户问题最相关的少量文档片段，再将这些片段作为上下文输入大语言模型生成回答。本平台 RAG 流水线采用三层混合检索策略：

层次	检索方式	成本	适用场景
Layer 1	元数据检索（文件名、路径、类型、日期）	零 AI 成本	精确文件定位
Layer 2	全文检索（SQLite FTS5）+ 语义向量检索（bge-m3 模型）	低成本	文件内容语义查询
Layer 3	视觉语言模型（VLM）按需处理图片/PDF	高成本，兜底	图像内容理解

MCP（Model Context Protocol）

MCP 是 Anthropic 提出的开放标准协议，规范化了大语言模型如何调用外部工具（Tool）的接口格式。本平台以 MCP 作为 Agent 与 NAS 文件系统工具之间的接口协议，主要收益为：工具定义与 LLM 提供商解耦，可切换不同的大模型（DeepSeek、Ollama 本地模型等）而无需修改工具实现；工具的输入输出 Schema 由 MCP 规范约束，减少模型产生格式错误的概率。

Agent Loop 设计

Agent 服务接收用户自然语言指令后，进入「感知 → 规划 → 工具调用 → 观察 → 回答」的循环：模型根据用户意图选择合适的工具（如 `list_files`、`search_files`、`read_file`），每次工具调用前经过 Permission Hook 向 authd 查询权限，通过后执行工具，将结果注入上下文，最多迭代 10 轮后生成最终答案。写操作（移动、删除）需要二次确认，防止误操作。

权限体系继承

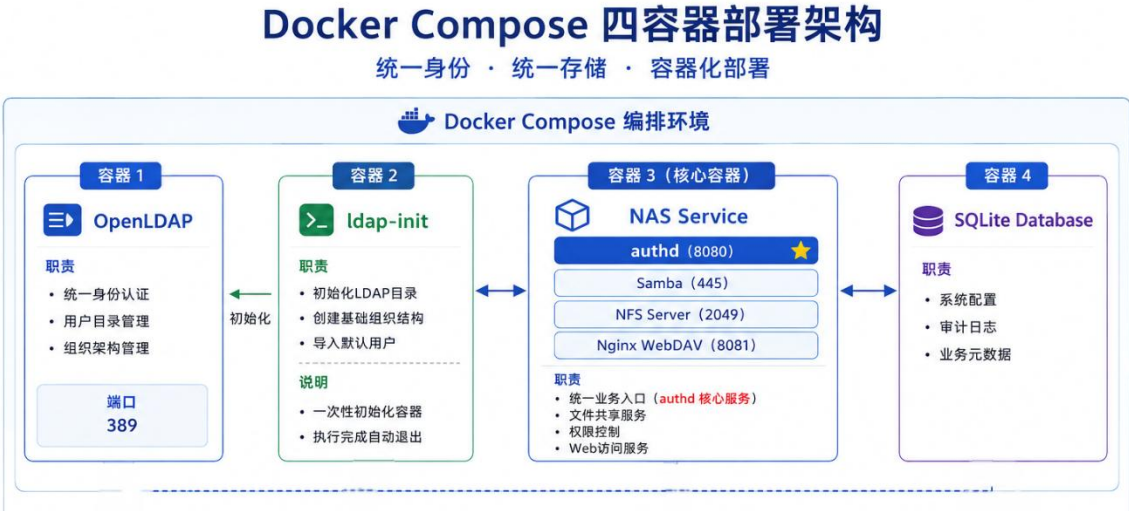
Agent 不自行维护权限逻辑，所有工具调用的权限决策完全委托给 authd。这保证了 AI 能力的扩展不会绕过平台统一的权限管控，Agent 用户在 NAS 上能做什么，Agent 就只能做什么，不多不少。

4.8 容器与部署技术栈

为什么采用容器化部署

NAS 平台涉及 OpenLDAP、Samba、NFS、Nginx、Go 后端等多个异构服务，各服务对系统库版本、配置文件路径有不同要求，传统裸机部署极易产生依赖冲突、环境差异导致的「在我机器上能运行」问题。容器化将每个服务及其依赖封装为独立镜像，保证了环境一致性：开发、测试、生产环境运行的是完全相同的镜像。

Docker Compose 四容器架构



4.9 安全技术栈

本平台安全能力由多个互补的安全机制层叠构成，形成深度防御体系。

JWT（会话安全）

HTTP API 层以 JWT（HS256）作为无状态会话载体，令牌有效期 24 小时。密钥通过环境变量注入，不硬编码于代码库。令牌包含用户名与角色声明，中间件在不查询后端的情况下完成身份核验。

LDAP Bind（认证安全）

用户密码验证通过 LDAP Bind 操作执行：将用户提供的密码与 LDAP 中存储的 SSHA 加盐哈希比对。密码永不以明文形式在网络或存储层出现。Samba 认证使用独立的 NT Hash（MD4），两种密码表示均存储于 LDAP，互不干扰。

POSIX ACL（访问控制安全）

文件系统层以 POSIX ACL 强制访问控制，任何越权访问在内核层面被拒绝，应用层无需额外判断。

路径穿越防护

所有涉及文件路径的 API 端点均在处理前执行路径合法性校验，阻断 `../` 等目录穿越攻击，防止用户访问其权限范围之外的文件系统路径。

SHA-256 哈希链（完整性安全）

审计日志通过 SHA-256 哈希链串联，每条记录的哈希值依赖前一条记录，形成不可静默篡改的链式结构。任何对历史记录的修改都会导致其后所有节点的哈希值失效，篡改行为可被自动检测。

NFC 认证（近场安全）

NFC 登录绑定基于 Android 设备的唯一标识（ANDROID_ID）与 NFC 标签 UID 的组合，绑定关系存储于 LDAP `ou=nfc_bindings`。NFC 标签本身不存储任何凭据，仅作为触发认证流程的物理媒介，防止标签数据泄露带来的安全风险。

设备签名（溯源安全）

每个证明包携带设备唯一标识（基于 `/etc/machine-id` 的 MD5 摘要），用于标识操作记录产生于哪台设备。规划中的 PUF 硬件签名将以 SM2 椭圆曲线算法将此标识与不可克隆的硬件物理特征绑定，进一步提升设备溯源的不可伪造性。

5 身份认证体系设计

5.1 设计理念

身份认证体系的核心设计原则是「Single Source of Truth」——全系统有且仅有一个权威用户信息源，任何协议的认证请求均指向同一源头。这一原则的实现载体是 OpenLDAP 目录服务。

5.2 OpenLDAP 统一身份源架构

OpenLDAP 目录按标准 X.500 树形结构组织，本平台的目录树结构如下：

```

dc=nas,dc=local
├── ou=users                                # 用户条目
│   ├── uid=alice                          # posixAccount + sambaSamAccount
│   ├── uid=bob
│   └── uid=admin
├── ou=groups                              # 用户组条目
│   ├── cn=nas-users                      # posixGroup (普通用户组)
│   └── cn=nas-admins                    # posixGroup (管理员组)
└── ou=nfc_bindings                       # NFC 绑定条目
    ├── cn=nfc_<tagUID_1>                 # NFC 标签 UID → 用户名 映射
    └── cn=nfc_<tagUID_2>

```

每个用户条目同时携带以下关键属性：

属性	来源 Schema	用途
uid / cn	core.schema	用户名与显示名
userPassword (SSHA)	core.schema	HTTP/WebDAV LDAP Bind 认证
uidNumber / gidNumber	posixAccount	NFS UID 映射
homeDirectory	posixAccount	用户主目录路径
sambaNTPassword (NT Hash)	sambaSamAccount	Samba SMB 认证
employeeType (admin/user)	inetOrgPerson	角色权限判断

5.3 用户注册与创建流程



5.4 多协议认证路径

访问协议	认证入口	认证方式	密钥来源
HTTP API	authd /api/login	LDAP Bind (userPassword)	LDAP SSHA
HTTP API (后续请求)	authd 中间件	JWT 签名验证	内存密钥
SMB (Windows)	Samba ldapsam	NTLM Challenge-Response	LDAP NT Hash
NFS (Linux)	Linux 内核 libnss-ldap	UID 数值映射	LDAP posixAccount
WebDAV	Nginx auth_request	HTTP Basic → authd LDAP Bind	LDAP SSHA
NFC 碰一碰	authd /api/nfc-login	NFC 标签 UID 查 LDAP 绑定表	LDAP ou=nfc_bindings

5.5 JWT 令牌设计

JWT 令牌结构：

```
Header: {"alg": "HS256", "typ": "JWT"}
Payload: {"username": "alice", "role": "user", "exp": <24h后时间戳>}
Signature: HMAC-SHA256(base64Header + "." + base64Payload, secretKey)
```

令牌有效期 24 小时，到期后客户端需重新登录获取新令牌。密钥通过容器环境变量注入，支持运维周期性轮换而无需重启服务。

5.6 开源组件与自研能力边界

能力	来源	说明
LDAP 目录存储与查询	OpenLDAP（开源）	目录服务核心，不做修改
LDAP Schema 扩展	社区 Schema（开源）	posix / samba / shadow
LDAP 客户端封装	自研（authd/ldap）	连接管理、UID 分配、Bind 封装
JWT 签发与验证	golang-jwt（开源） + 自研封装	自研部分负责密钥管理与声明定义
NFC 绑定逻辑	自研（authd/handler/nfc.go）	标签 UID 与用户映射，存 LDAP
统一注册/登录接口	自研（authd）	跨协议身份协调的业务入口

6 权限控制体系设计

6.1 设计理念

权限体系的核心设计原则是「Single Permission Source」——全系统有且仅有一套权限规则，存储于 Linux 文件系统的 POSIX ACL 中，通过内核强制执行，任何协议的文件访问均受同一套规则约束。这一设计消除了多权限副本之间的漂移风险。

6.2 POSIX ACL 与传统 Unix 权限的区别

传统 Unix 文件权限（rwxrwxrwx）只能为三个主体（文件所有者、所属组、其他人）分别设置读/写/执行权限，无法为某个特定用户单独授权，无法满足企业多人协作场景的精细化需求。

POSIX ACL 在此基础上引入了可扩展的 ACL 条目（ACE，Access Control Entry），可为任意具名用户或组单独设置权限：

```
文件： /data/alice/project-report.pdf
ACL 条目：
user::rwx          (所有者 alice，完全控制)
group::---          (所属组，无权限)
user:bob:r--        (bob 单独被授予只读权限)
user:charlie:rw-     (charlie 单独被授予读写权限)
mask::rw-           (有效权限掩码)
other::---           (其他所有人，无权限)
```

6.3 为什么采用 POSIX ACL

POSIX ACL 选择的核心理由在于其是操作系统内核的原生能力：Samba、NFS、VFS 层均原生支持 POSIX ACL，无需任何桥接或转换层。一次 `setfacl` 调用的变更立即对所有协议可见，无延迟，无同步，不会漂移。相比自研权限表的方案，引入的代码量更少，可靠性更高，维护成本更低。

6.4 权限结构设计

用户私有空间

每个用户在 `/data/<username>` 下拥有私有目录，初始权限设置为 `700`（仅所有者可读写执行），其他任何用户默认无法访问。

管理员特权

管理员用户（`employeeType: admin`）在 HTTP API 层享有超级权限：可列举所有用户的目录，可对任意路径执行文件操作，可管理所有用户账号。在文件系统层，管理员账号被赋予相应目录的 ACL 条目，使系统调用层面的权限与 API 层面的权限声明一致。

共享授权机制

用户可通过 HTTP API 的共享权限接口，将自己的文件或目录授权给其他特定用户：

```
POST /api/share/permission
{
  "path": "/data/alice/project/",
  "target_user": "bob",
  "permission": "r"    // r=只读, rw=读写, rwx=完全控制
}
```

authd 接收请求后，校验操作者对该路径的所有权，通过后调用 `setfacl` 写入 ACL 条目。

跨协议权限一致性保障



6.5 权限角色矩阵

操作	普通用户（自己目录）	普通用户（他人目录）	管理员
列目录	✓	✗（除非被共享）	✓（任意）
上传文件	✓	✗（除非被共享 rw）	✓（任意）
下载文件	✓	✗（除非被共享 r）	✓（任意）
删除文件	✓	✗	✓（任意）
设置共享	✓（自己的文件）	✗	✓（任意）

操作	普通用户（自己目录）	普通用户（他人目录）	管理员
查看系统统计	✗	✗	✓
管理用户账号	✗	✗	✓
查看审计日志	✗	✗	✓
导出存证包	✗	✗	✓

6.6 开源组件与自研能力边界

能力	来源	说明
ACL 存储与执行	Linux 内核（开源）	内核 ACL 子系统，不做修改
setfacl / getfacl 工具	acl 软件包（开源）	系统调用封装工具
路径合法性校验	自研（system/files.go）	防目录穿越、路径归一化
权限管理 API	自研（handler/permission.go）	ACL 读写接口、共享逻辑
角色权限决策	自研（authd 中间件）	admin/user 角色路由守卫

7 文件服务体系设计

7.1 多协议融合架构设计理念

不同类型的终端设备和应用场景对文件访问协议有不同偏好，没有任何单一协议能无缝覆盖所有场景。本平台采用多协议融合架构：在同一套文件存储与权限体系之上，并行提供四种访问协议，以统一的底层保障跨协议的数据一致性与权限一致性。

7.2 SMB（Server Message Block）

设计目标：为 Windows 客户端提供原生、无感知的文件共享体验，无需安装任何第三方客户端。

适用场景：企业 Windows 工作站、Windows Server 环境、Office 文档在线编辑、Windows 备份目标。

技术原理：Samba 实现了 SMB/CIFS 协议栈，Windows 客户端通过 \\<NAS_IP>\<share> 路径挂载后，即可像操作本地磁盘一样操作 NAS 文件。认证采用 NTLM 协议，Samba 的 ldapsam 后端直接向 OpenLDAP 查询 sambaNTPassword 属性进行口令验证，无需独立用户数据库。文件访问权限由

Linux 内核的 POSIX ACL 控制，Samba 尊重 ACL 设置，不引入额外权限层。

优势：Windows 原生支持，用户体验最佳；与 LDAP 身份源直接集成；权限与其他协议完全统一。

局限性：SMB 协议设计复杂，防火墙通常需要开放 445 端口；在 macOS 和 Linux 上的兼容性相对 NFS/WebDAV 稍弱；协议加密依赖 SMB3，旧版客户端需额外配置。

7.3 NFS (Network File System)

设计目标：为 Linux/Unix 系统提供高性能、低延迟的网络文件挂载能力，满足服务器级文件共享需求。

适用场景：Linux 服务器、CI/CD 构建系统、容器持久化存储、视频监控写入节点。

技术原理：NFS 通过 UID/GID 数值实现权限判断，内核在客户端和服务端均以相同的 UID 数值识别用户身份。本平台通过 libnss-ldap 使 NAS 主机的用户名解析 (getpwnam、getpwuid) 指向 OpenLDAP，并引入 Kerberos 作为底层身份认证协议，确保 NFS 客户端的用户身份在接入前已经过 KDC 的强认证。当 Kerberos 认证通过后，客户端获取对应的 Kerberos 票据，并以该票据所标识的用户身份发起 NFS 访问；服务端内核在收到请求后，结合 Kerberos 映射的用户主体，通过 LDAP 查询该用户对应的 UID/GID 信息及 POSIX 属性。例如，当一个经过 Kerberos 认证的客户端以用户 alice 的身份访问文件时，服务端通过 LDAP 获知该用户对应的 UID 为 2001，进而以此 UID 应用 alice 的 POSIX ACL 权限，实现统一、安全且可审计的权限控制。

优势：内核级挂载，性能最优；无客户端软件依赖；适合高吞吐写入场景。

局限性：跨互联网使用安全性较低，通常限于局域网；NFSv3 无内置加密，生产环境建议叠加 VPN 或 Kerberos；Windows 客户端 NFS 支持有限。

7.4 WebDAV

设计目标：为跨平台客户端 (macOS Finder、Linux davfs2、Rclone、文档编辑工具) 提供基于 HTTP 的文件访问能力，覆盖 SMB/NFS 覆盖不到的场景。

适用场景：macOS 客户端、跨平台文档协作工具 (Obsidian、Joplin 等)、云存储备份工具 (Rclone)、企业 OA 系统附件存储。

技术原理：WebDAV 扩展 HTTP 协议，新增 PUT (上传)、DELETE (删除)、MKCOL (创建目录)、COPY (复制)、MOVE (移动)、PROPFIND (查询属性) 等方法。本平台以 Nginx 的 ngx_http_dav_module 模块提供 WebDAV 服务，监听 8081 端口。身份认证采用 HTTP Basic Auth，Nginx 的 auth_request 指令将每次请求的认证委托给 authd 的 /internal/verify-password 接口：Nginx 接收到请求后，先向 authd 发起子请求验证凭据，authd 通过 LDAP Bind 验证成功后返回 200，Nginx 才放行原始请求。

优势：基于 HTTP，穿透防火墙能力强；macOS 和 Linux 原生支持；无需安装专用客户端。

局限性：性能低于 SMB/NFS（HTTP 协议开销更大）；目录列举（PROPFIND）对大量文件时延迟较高；标准 WebDAV 不支持追加写入，大文件断点续传需客户端额外支持。

7.5 HTTP RESTful API

设计目标：为移动 App、Web 管理台和程序化客户端提供基于 JSON 的结构化文件访问接口，支持精细化权限控制与元数据查询。

适用场景：Android 移动端文件管理、Web 管理后台文件浏览器、自动化脚本、AI Agent 工具调用。

技术原理：authd 通过 Gin 框架暴露 RESTful 文件 API，支持目录列举、文件上传（multipart）、文件下载（流式响应）、删除、移动、创建目录等操作。所有请求需携带有效 JWT 令牌，中间件在请求进入处理逻辑前完成身份核验与角色判断。文件系统操作前执行路径合法性校验，阻断越权访问。所有文件操作在执行后同步写入审计日志。

优势：JSON 接口，对 Web/移动/脚本客户端最友好；支持 JWT 无状态认证；与审计系统深度集成；适合程序化工具调用。

局限性：不适合 OS 级文件系统挂载（无法像本地磁盘一样使用）；文件上传需完整加载入内存，大文件需特别处理；需要 HTTP 客户端库支持，不如 SMB/NFS 对终端透明。

7.6 多协议融合的技术价值

多协议并存的根本价值在于：NAS 平台的服务能力不因终端操作系统的差异而缩减。企业内同时存在 Windows 工作站（SMB）、Linux 服务器（NFS）、macOS 笔记本（WebDAV）和移动设备（HTTP API）时，所有终端均能以其最优方式接入同一个文件系统，访问同一份数据，受同一套权限管控。这是传统单协议 NAS 无法实现的能力。

8 自研核心服务设计

8.1 authd 的核心定位

authd（Authentication Daemon）是本平台最核心的自研服务，是整个系统的统一业务入口。它不是一个简单的 HTTP 代理，而是承担了身份认证、权限决策、文件服务、审计记录、系统管理五大职责的中枢服务。平台上所有来自 Web 管理台和移动 App 的业务请求，均由 authd 统一接收与处理。

8.2 设计目标

authd 的设计目标是：在一个进程内，以最小的依赖完成 NAS 核心业务逻辑，使系统整体可运行于资源受限的边缘设备之上，同时保证各能力模块的清晰边界与可独立演进性。

8.3 模块职责划分



8.4 统一业务入口设计思想

将所有 HTTP 业务逻辑集中于 authd 而非分散为多个微服务，这是本平台在当前阶段的明确架构选择。其理由如下：

降低运营复杂度：NAS 设备通常由非专业运维人员管理，单进程部署比微服务集群更易排查故障、更易备份、更易迁移。

减少网络跳数：认证、权限判断、文件操作在同一进程内完成，避免微服务间 RPC 调用的网络延迟，提升响应速度。

一致的事务语义：文件操作与审计日志写入在同一进程内顺序完成，不存在跨服务的分布式事务问题，日志不会因服务间通信失败而丢失。

适合边缘资源约束：单进程 Go 服务在低配 ARM 硬件上的内存占用可控制在数十 MB 级别，远低于微服务架构的基础开销。

8.5 authd 与各模块的协作关系

```
OpenLDAP ← authd (身份操作: Bind/Add/Modify/Delete)
Linux ACL ← authd (权限操作: setfacl/getfacl via system call)
SQLite ← authd (审计操作: INSERT certified_operations / proof_records)
Linux FS ← authd (文件操作: Read/Write/Delete/Move via os package)
Nginx → authd (WebDAV 认证回调: auth_request → /internal/verify-password)
Agent → authd (权限校验回调: /internal/check-access, 规划中)
Web/App → authd (所有业务请求入口: 8080 端口)
mDNS ← authd (服务广播: 启动时注册 _nas._tcp.local.)
```

8.6 统一业务入口带来的架构收益

通过将业务逻辑集中于 authd，平台获得了以下可量化收益：

- **审计完整性：**所有经过 authd 的操作均被记录，无法绕过审计层访问文件系统（SMB/NFS 绕过 authd 的操作通过文件系统审计模块补充覆盖，规划中）。
- **权限一致性：**HTTP API 的权限决策逻辑只有一份，不存在多处维护导致的策略分歧。
- **可观测性：**所有请求在 authd 的中间件链上留有日志，便于问题排查与性能分析。
- **可扩展性：**新增 API 能力只需在 authd 内增加 Handler，无需部署新服务，扩展成本极低。

9 智能服务体系设计

9.1 设计背景与目标

传统 NAS 提供的是「文件的存放与取用」，用户必须知道文件在哪里才能找到它。智能服务体系的设计目标是将 NAS 转化为「可理解内容的知识库」：用户可以用自然语

言描述需求 ("找出去年所有关于采购合同的文件")，系统自动完成语义理解、内容检索、结果汇总，并以结构化答案返回。

9.2 智能服务链路



9.3 Agent 设计

Agent 服务是智能能力的总协调者，负责接收用户指令并驱动工具调用循环（Agent Loop）：

Prompt Builder：将用户指令与当前上下文（用户身份、历史轮次、可用工具列表）组装为发送给大语言模型的提示词。

LLM 调用：将组装好的提示词发送给大语言模型（当前支持 DeepSeek，规划本地 Ollama），获取工具调用意图或最终答案。

Tool Dispatch：解析模型返回的工具调用请求，路由至对应的工具处理函数。

Permission Hook：每次工具调用前，向 authd 查询当前用户对目标资源的访问权限。权限不足则直接返回拒绝，不执行工具。

结果注入：将工具执行结果注入对话上下文，触发下一轮模型调用，直至模型产生最终答案或达到最大迭代轮数（10 轮）。

写操作保护：文件移动、删除等写操作在执行前向用户发出二次确认请求，防止模型理解偏差导致误操作。

9.4 MCP 协议的选择理由

MCP (Model Context Protocol) 由 Anthropic 提出并开源，是当前 LLM 工具调用领域最具生态影响力的标准协议。采用 MCP 的核心理由：

- **LLM 提供商无关**：工具定义与 LLM 厂商解耦，可无缝切换 DeepSeek、Ollama、Claude 等不同模型，无需修改工具实现代码。
- **Schema 约束**：MCP 要求工具以 JSON Schema 声明输入输出格式，减少模型产生格式错误的概率，提升 Agent Loop 的稳定性。
- **生态兼容**：采用 MCP 标准意味着平台工具可被任何支持 MCP 的 AI 客户端直接调用，扩大了潜在集成范围。

9.5 RAG 三层混合检索策略

本平台 RAG 流水线采用三层递进检索策略，在成本与召回质量之间取得平衡：

Layer 1 — 元数据检索（零 AI 成本）

基于文件名、路径、创建时间、文件大小、MIME 类型等结构化属性进行精确匹配与范围过滤。绝大多数「找文件」类请求（"找一下上个月的 PDF 文件"）在此层即可完成，无需调用 AI 模型，响应延迟极低。

Layer 2 — 内容混合检索（低成本 AI）

对需要理解文件内容的查询，进入第二层：

- **SQLite FTS5 全文检索**：基于倒排索引快速定位包含关键词的文档片段，适合精确词语匹配。
- **bge-m3 语义向量检索**：使用向量化模型将查询语句转化为语义向量，与预建的文件内容向量库进行相似度匹配，支持近义词、同义表达的语义召回。
- 两路检索结果融合排序后返回最相关的文档片段作为 LLM 上下文。

Layer 3 — 视觉语言模型按需处理（高成本兜底）

对图片、PDF 扫描件等无法通过文本检索理解内容的文件，按需调用视觉语言模型（VLM）进行多模态内容理解。由于此层成本较高，仅在前两层无法满足检索需求时触发。

9.6 权限体系继承

智能服务体系不自行维护权限逻辑，完全继承 authd 的权限体系。这一设计保证了 AI 能力扩展的安全边界：Agent 代表某用户执行工具调用时，所能访问的文件范围与该用户通过 HTTP API 直接访问时完全一致，不因引入 AI 中间层而产生权限放大或缩小。

10 移动接入与设备发现设计

10.1 设计目标

移动接入体系的设计目标是：让用户在局域网内以「零配置、零输入」的方式连接 NAS 设备，并以「碰一碰」的近场交互完成登录认证，将移动端接入 NAS 的操作复杂度降至最低。

10.2 三级连接回退机制

移动端连接 NAS 的核心挑战是：IP 地址可能变化（DHCP 重新分配），且用户不应关心 IP 地址。本平台设计了三级自动回退连接机制：



Level 1 — WiFi P2P 直连

通过 Android WiFi P2P (Wi-Fi Direct) 技术在手机与 NAS 之间建立点对点 WiFi 连接, 无需接入点。P2P 连接建立后, NAS 作为 P2P 组长 (Group Owner), 固定 IP 为 192.168.49.1, App 直接连接该地址。此能力由 Kotlin 原生模块 (WifiP2pModule) 实现, 通过 JSB 暴露给 React Native 层。

Level 2 — mDNS 自动发现

authd 启动时通过 grandcat/zeroconf 库向局域网广播 `_nas._tcp.local.` 服务记录, 携带设备名称、IP 地址和端口信息。移动 App 启动后自动扫描此服务记录, 无需用户输入任何网络参数即可找到 NAS 设备。该机制适用于 99% 的家庭和办公室局域网场景。

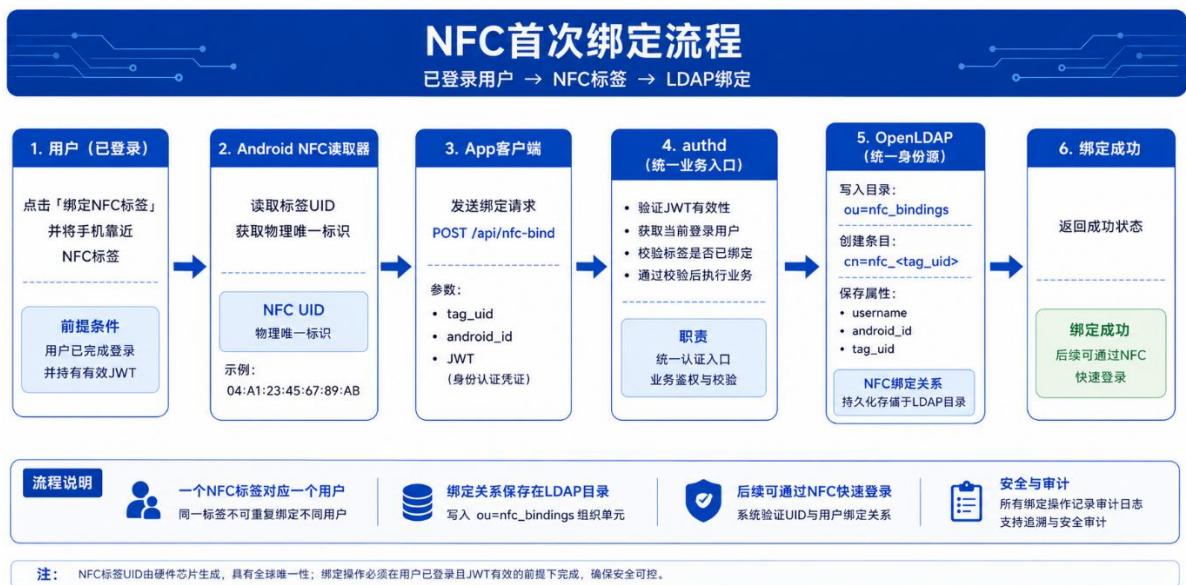
Level 3 — 缓存 IP 验证

每次成功连接后, App 将服务器 IP 持久化到 AsyncStorage。下次启动时若 mDNS 发现失败 (如 NAS 设备尚未广播), 则尝试连接上次缓存的 IP。适用于设备 IP 未变化但 mDNS 发现暂时失败的场景。

10.3 NFC 碰一碰登录设计

设计理念: NFC 登录的核心价值是消除密码输入操作, 用物理近场接触代替凭据输入, 适合家庭场景下的便捷登录, 以及企业场景下无需记忆密码的快捷接入。

首次绑定流程:



后续免密登录流程:



安全设计：NFC 标签本身不存储任何凭据（密码、密钥），仅作为触发认证流程的物理媒介。绑定关系以 ANDROID_ID 进行设备绑定，即使 NFC 标签被复制，在原绑定设备之外也无法用于登录。绑定数据存储在 LDAP，与用户身份数据同源管理。

10.4 移动端架构设计

框架分层：



JS 层负责业务逻辑与 UI 渲染，原生模块层负责访问 Android 系统 API。两层之间通过 JSB 机制通信：JS 层调用原生模块的方法，Kotlin 层执行系统调用后将结果回调给 JS 层。这种分层保证了跨平台业务代码与平台原生能力的解耦。

11 审计日志与可信存证体系

11.1 设计目标

可信存证体系的建立源于三类核心需求：

- **合规审计需求**：企业监管环境要求系统能够回答「谁在什么时间对什么文件做了什么操作」，且答案必须可信。
- **操作追溯需求**：文件误删、数据泄露等事件发生后，能够精确还原事件链路，确定责任主体。
- **不可篡改需求**：操作记录必须具备防篡改能力——即使数据库管理员也无法静默修改历史记录而不留痕迹。

11.2 两表协作设计

系统通过两张相互配合的数据库表实现可信存证：

certified_operations（操作记录表）

记录每一次文件操作与身份操作的完整上下文，包含 22 个字段：

字段类别	字段内容
操作标识	操作 ID、时间戳、操作类型、操作动作
主体信息	操作用户名、用户 UID
客体信息	文件路径、目标路径（移动操作）、文件名、是否目录
文件属性	文件大小、MIME 类型、所有者 UID/名称、所属组、文件权限、修改时间
内容指纹	文件内容哈希值、哈希算法（SHA-256，规划升级 SM3）

proof_records（哈希链节点表）

以链式哈希结构串联所有操作记录，保证记录的不可篡改性：

字段	含义
id	节点序号

字段	含义
cert_id	关联的 certified_operations 记录 ID
chain_index	在哈希链中的位置索引
prev_hash	前一个节点的哈希值
data_hash	本节点对应操作记录的哈希值
signature	设备签名（当前为设备 ID，规划升级为 PUF SM2 签名）
device_uid	产生此记录的设备唯一标识
sig_timestamp	节点生成时间戳
hash_algo	哈希算法标识

11.3 哈希链原理

哈希链的核心思想是：每个节点的哈希值依赖于前一个节点的哈希值，形成因果锁链：

节点 N 的哈希 = SHA-256（节点 N-1 的哈希 + 节点 N 的数据哈希）

创世节点（N=0）：prev_hash = "0000...0000"（固定初始值）

节点 1：node_hash_1 = SHA-256(prev_hash_0 + data_hash_1)

节点 2：node_hash_2 = SHA-256(node_hash_1 + data_hash_2)

节点 N：node_hash_N = SHA-256(node_hash_{N-1} + data_hash_N)

如果攻击者篡改了节点 K 的数据，则 data_hash_K 发生变化，导致 node_hash_K 变化，进而导致节点 K+1 至最新节点的所有哈希值均失效。完整性验证时，系统重新计算全链哈希并与存储值比对，任何篡改均可被检测。

11.4 存证包导出与独立验证

管理员可通过 `GET /api/proof/bundle` 接口导出标准格式的存证包（JSON），包含：

```
{
  "device_id": "<设备唯一标识>",
  "export_time": "<导出时间戳>",
  "total_records": "<记录总数>",
  "chain_valid": true,           ← 后端在导出时实时重算全链，验证结果
  "records": [...]             ← 完整的操作记录与哈希链节点
}
```

存证包可离线保存，任何持有验证算法（SHA-256 或规划中的 SM3）的第三方均可独立重算哈希链，验证记录完整性，无需依赖本平台。

11.5 可信存证体系的业务价值

- **法律举证**：存证包可作为法律程序中的电子证据辅助材料，证明特定操作发生于特定时间。
- **安全事件响应**：发生数据泄露或误删事件时，能够精确回溯操作链路，快速定位责任节点。
- **监管合规**：满足 ISO 27001、等保 2.0 等标准对操作日志不可篡改性要求。
- **内部审计**：IT 管理员可定期导出存证包，作为内部合规审计的审计留底材料。

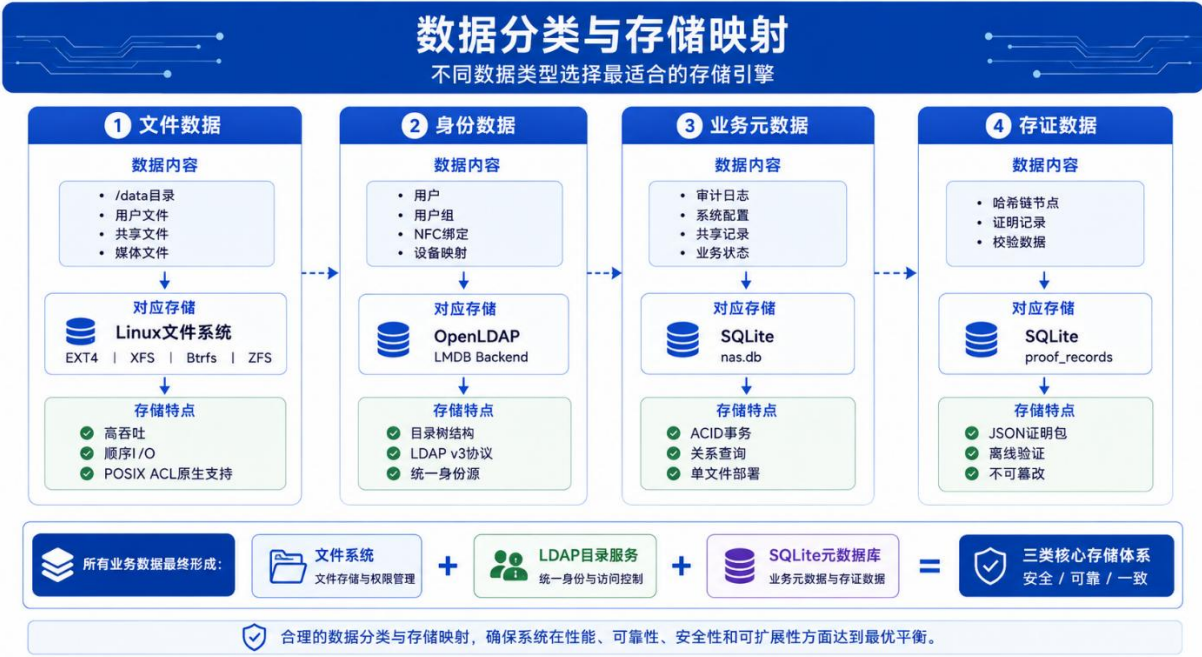
12 数据存储架构设计

12.1 数据分类与存储映射

本平台将系统数据按用途、访问特征、安全要求分为四类，分别由最适合的存储系统承载：

12.2 文件数据层

存储路径： `/data/` （容器内路径，通过 Docker Volume 映射至宿主机持久存储）



目录结构：

```
/data/  
├─ shared/           # 公共共享目录（所有用户可读，适合共享资源分发）  
├─ <username_1>/     # 用户私有目录（初始 mode: 700，仅本人可访问）  
├─ <username_2>/  
├─ ...  
└─ .nas.db           # SQLite 数据库文件（与文件数据共享 volume）
```

文件系统选型策略：平台不强绑定特定文件系统，运维人员可根据硬件条件与业务需求选择：EXT4 适合大多数通用场景；XFS 适合大文件高吞吐写入（视频监控）；Btrfs 适合需要快照与内置 RAID 的场景；ZFS 适合对数据完整性要求极高的企业场景。

ACL 与文件系统的关系：POSIX ACL 数据作为文件系统扩展属性（xattr）存储在文件系统中，与文件数据不可分离。文件系统迁移（如从 EXT4 迁移至 XFS）时，需显式保留 xattr，否则 ACL 数据丢失。

12.3 身份数据层

存储系统：OpenLDAP，内部使用 LMDB（Lightning Memory-Mapped Database）引擎

数据持久化：通过 Docker Volume（ldap-data）挂载，容器重建时数据保留

数据内容：

OU	存储内容	典型访问模式
ou=users	用户账号（属性集：posixAccount + sambaSamAccount）	登录时 Bind，注册时 Add
ou=groups	用户组（posixGroup）	NFS 组权限解析时查询
ou=nfc_bindings	NFC 标签绑定关系	NFC 登录时查询

备份策略：LDAP 数据库可通过 slapcat 工具导出为 LDIF 格式的纯文本备份，人类可读，可版本控制，可跨版本恢复。

12.4 业务元数据层

存储系统：SQLite，文件路径 /data/.nas.db

表结构设计：

```
certified_operations          ← 操作审计日志（22 字段）
    索引：timestamp DESC, type, user_name

proof_records                 ← 哈希链节点（9 字段）
    索引：chain_index ASC, cert_id
```

SQLite 选型理由：

- 零配置：无独立服务进程，随应用启动，无端口、无守护进程
- 单文件：整个数据库为一个文件，备份即复制，迁移零成本
- ACID：完整事务语义，操作日志写入与哈希链节点生成保持原子性
- 纯 Go 实现（modernc）：无 CGO 依赖，保证后端二进制静态编译

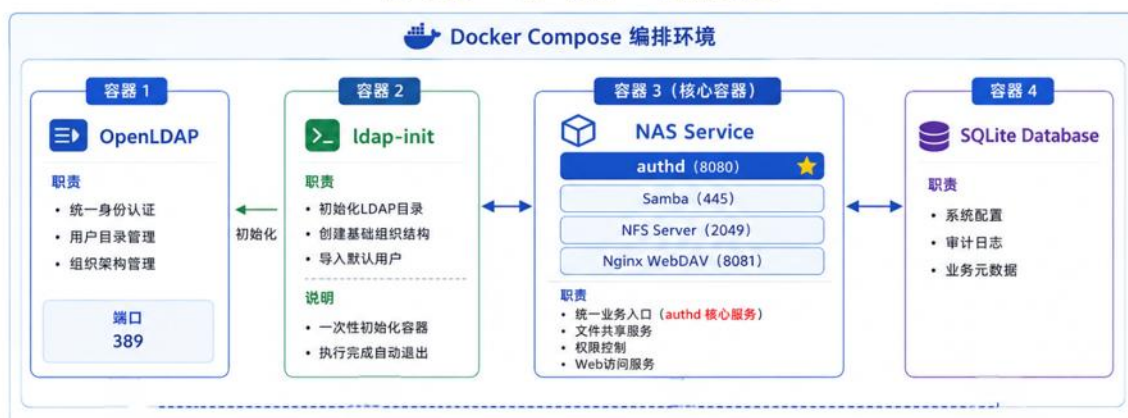
13 网络与容器架构设计

13.1 整体容器架构

本平台以 Docker Compose 编排四个容器，共同构成完整的 NAS 服务栈：

Docker Compose 四容器部署架构

统一身份 · 统一存储 · 容器化部署



13.2 容器职责划分

openldap 容器

承载 OpenLDAP 目录服务，是身份数据的持久化存储。以 `osixia/openldap:1.5.0` 为基础镜像，通过环境变量注入域名 (`dc=nas,dc=local`) 与管理员密码，数据通过两个 Volume 持久化 (`ldap-data` 存储目录数据，`ldap-config` 存储服务配置)。该容器是 nas 主容器的依赖项，需先启动并就绪。

ldap-init 容器

一次性初始化容器，在 openldap 容器就绪后执行 LDIF 初始化脚本，导入预设的组织单元 (`ou=users`、`ou=groups`、`ou=nfc_bindings`) 与初始管理员账号。初始化完成后容器自动退出，不占用持续资源。

nas 主容器

系统核心容器，内部运行四个服务进程：

服务	启动方式	依赖
authd	Go 编译的静态二进制	OpenLDAP、SQLite
Samba	samba 系统服务	OpenLDAP (ldapsam)
NFS Server	nfs-kernel-server 系统服务	Linux 内核、libnss-ldap
Nginx	nginx 系统服务	authd (auth_request 回调)

容器启动脚本（`start.sh`）按顺序启动各服务，确保依赖链的正确初始化顺序。

13.3 网络拓扑设计

network_mode: host 选择理由

SMB（445）、NFS（2049）等协议对底层网络特性有特殊依赖：

- SMB 协议使用 NetBIOS/mDNS 进行局域网设备发现，依赖广播和多播，这在 Docker 的 bridge 网络模式下无法直接工作。
- NFS 协议的 RPC 端口映射（portmapper）依赖宿主机端口。
- Samba 在 bridge 模式下需要复杂的端口映射配置，稳定性差。

采用 `host` 网络模式，容器直接使用宿主机的网络接口和 IP 地址，消除 NAT 层，各协议服务的网络行为与裸机部署完全一致。

服务间通信

nas 主容器内的四个服务通过 `localhost` 进行进程间通信：Nginx 通过 `localhost:8080` 回调 `authd`；各服务通过 `localhost:389` 连接 OpenLDAP（因 `network_mode: host`，`openldap` 容器的 389 端口直接绑定在宿主机上）。

13.4 数据卷设计

卷名称	挂载路径（容器内）	用途	持久化策略
ldap-data	/var/lib/ldap	LDAP 目录数据库（用户、用户组、NFC 绑定等目录数据）	永久保留
ldap-config	/etc/ldap/slapd.d	LDAP 服务配置（Schema、ACL、索引配置等）	永久保留
nas-data	/data	用户文件存储、共享文件、SQLite 数据库（nas.db、proof.db）	永久保留

所有 Volume 均为 Docker Named Volume（命名卷），与容器生命周期解耦。执行 `docker compose down` 不会删除数据，执行 `docker compose down -v` 才会删除 Volume（需谨慎操作）。

13.5 服务依赖与启动顺序



13.6 为什么采用容器化部署

环境一致性：将 OpenLDAP、Samba、NFS、Nginx、Go 服务及其所有依赖（libnss-ldap、libpam-ldap、acl 等）封装在镜像中，开发、测试、生产环境运行完全相同的制品，消除「在我机器上能跑」的环境差异问题。

部署简洁性：整个平台通过 `docker compose up -d` 单条命令启动，无需逐一配置各服务的安装、路径、依赖。对于非专业运维人员管理的 NAS 设备，部署复杂度极低。

升级隔离性：服务升级时，只需更新对应容器镜像，不影响宿主机系统环境，回滚也只需切换镜像版本。

资源隔离：每个容器有独立的文件系统视图，减少服务间的意外相互影响。

14 安全架构设计

14.1 安全架构概述

本平台的安全架构采用纵深防御（Defense in Depth）策略，在身份、访问、数据、审计四个层面分别部署安全机制，任意单一机制被绕过不会导致整体安全失效。



14.2 身份安全

密码存储安全：用户密码永不以明文存储。HTTP/WebDAV 认证使用 SSHA（加盐 SHA-1）哈希存储于 LDAP；Samba SMB 认证使用 MD4 NT Hash 存储于 LDAP。两种表示互相独立，修改一种不影响另一种。

LDAP Bind 认证：密码验证通过 LDAP Bind 操作执行，由 OpenLDAP 服务端比对哈希，密码明文不经过 authd 内存（仅在传输层短暂存在），降低中间人截获风险。

账号创建安全：用户注册时，UID 从 2000 起自增分配，与 Linux 系统账号（UID 0-999）和服务账号（UID 1000-1999）保持隔离，防止特权 UID 被占用。

14.3 访问控制安全

JWT 安全设计：

- 算法：HMAC-SHA256（HS256），对称签名，验证快速
- 有效期：24 小时，平衡安全性与用户体验
- 密钥管理：通过环境变量注入，不硬编码于代码，支持运维周期性轮换

- 声明设计：Payload 包含 username 与 role，中间件无需查询后端即可完成授权决策

角色守卫：管理员专属接口（/api/dashboard/*、/api/users/*、/api/logs/*、/api/proof/*）由独立中间件守卫，普通用户 JWT 携带 role: user 时直接返回 403，不进入业务处理逻辑。

接口隔离：内部接口（/internal/*）仅供容器内其他服务调用（Nginx auth_request），通过网络层限制（仅 localhost 可达）实现与外部网络的隔离。

14.4 数据安全

路径穿越防护：所有接收文件路径参数的接口，在处理前均执行路径合法性校验（ValidatePath）：规范化路径后检查是否在允许根目录（/data）下，拒绝包含 ../ 等穿越序列的路径，防止用户访问系统文件或其他用户的私有目录。

POSIX ACL 强制访问控制：文件系统操作由 Linux 内核的 ACL 子系统强制执行，应用层的权限判断失效时，内核层仍会拒绝未授权的文件系统调用，形成双重防护。

目录权限初始化：用户主目录创建时设置为 mode: 700，确保其他用户默认无任何访问权限，需显式授权后方可访问。

文件内容哈希：上传文件时计算 SHA-256 内容哈希并存入 certified_operations，用于后续验证文件内容是否被篡改，同时作为存证的内容指纹。

14.5 审计安全

不可篡改日志：操作日志通过哈希链结构保证不可静默篡改。任何对历史记录修改均导致链上后续所有节点的哈希值失效，完整性校验可自动检测篡改。

操作全覆盖：所有经过 authd 的业务操作（文件上传、下载、删除、移动、用户注册、登录）均被记录，无白名单豁免机制。

设备溯源：每条证明链节点携带设备唯一标识，证明包可溯源至产生记录的具体设备，防止日志被跨设备伪造。

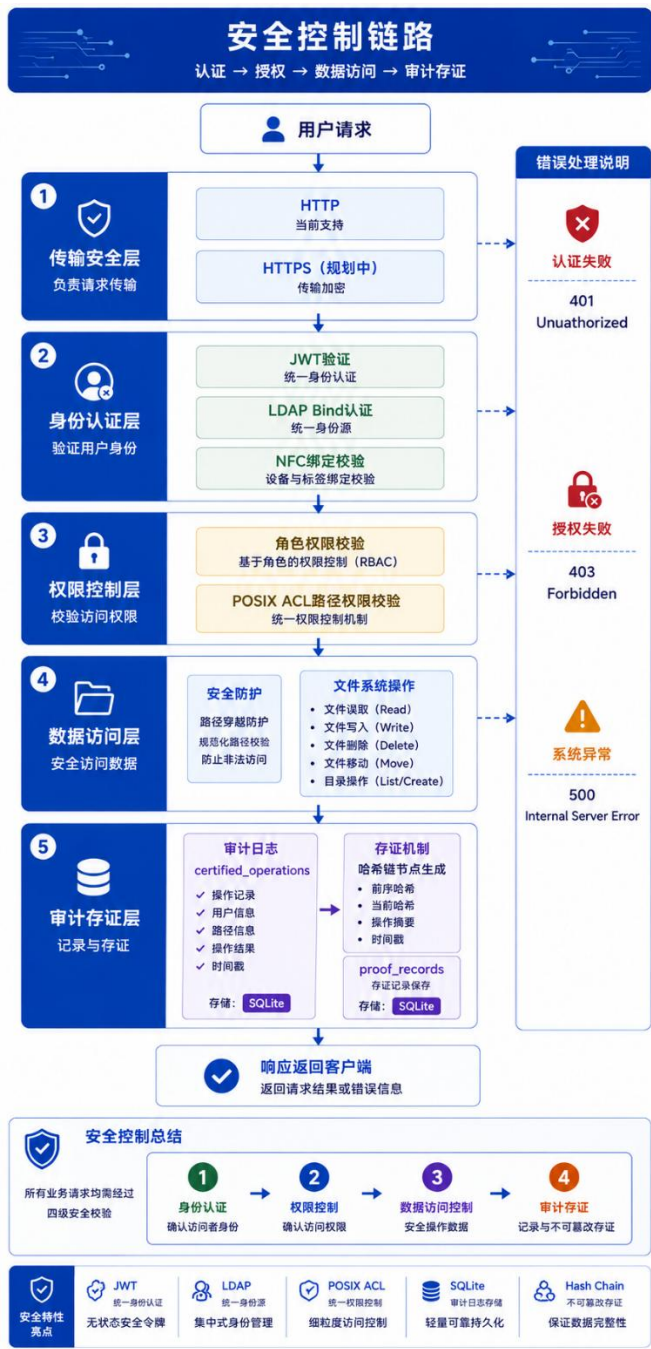
14.6 NFC 认证安全

标签无凭据原则：NFC 标签仅存储 UID（物理唯一标识），不存储任何密码、密钥或 Token。标签丢失或被复制，攻击者无法从标签本身提取凭据。

设备绑定机制：NFC 绑定关系存储 ANDROID_ID（设备级唯一标识），NFC 登录时同时验证标签 UID 与 ANDROID_ID。即使攻击者复制了 NFC 标签，在未授权设备上碰触也无法通过认证。

绑定管理：绑定关系存储于 LDAP ou=nfc_bindings，管理员可随时通过 Web 管理平台查看并撤销任意用户的 NFC 绑定，响应设备丢失等安全事件。

14.7 安全闭环



每个请求在身份、授权、数据、审计四个关卡均经过安全检查，任一关卡拒绝即终止请求，所有成功与失败操作均留有审计记录。

15 开源组件与自研模块说明

15.1 开源组件清单

模块名称	类别	技术来源	承担职责	核心价值
OpenLDAP	开源	OpenLDAP Foundation	统一身份目录服务、用户/组/NFC 绑定存储	原生支持 POSIX UID 映射与 Samba ldapsam, 无缝跨协议身份联动
Samba	开源	Samba Team	SMB/CIFS 文件共享服务	Windows 原生文件共享协议实现, ldapsam 直连 LDAP
NFS Server	开源	Linux 内核	NFS 网络文件系统服务	Linux 原生高性能网络文件协议
Nginx + DAV 模块	开源	Nginx Inc.	WebDAV 文件访问服务、反向代理	高性能 HTTP 服务器, 原生支持 auth_request 认证委托
Docker + Compose	开源	Docker Inc.	容器运行时与多容器编排	服务隔离、环境一致性、单命令部署
SQLite (modern)	开源	SQLite / Enzo Ruiz	业务元数据与审计数据存储	零依赖单文件数据库, ACID 事务, 纯 Go 实现无 CGO
Go 1.25	开源	Google / Go Team	后端核心开发语言	高并发低资源、静态编译、跨平台
Gin v1.9	开源	gin-gonic	HTTP 路由框架与中间件	高性能路由, 中间件链, Swagger 集成
golang-jwt	开源	golang-jwt	JWT 令牌签发与解析	HS256 无状态会话, 标准实现
go-ldap	开源	go-ldap	Go LDAP 客户端	完整 LDAP v3 操作封装, 连接池支持
zeroconf	开源	grandcat	mDNS/Zeroconf 服务发现	局域网零配置设备广播
gopsutil	开源	shirou	系统资源监控 (CPU/内存/磁盘)	跨平台系统指标采集
Next.js 16	开源	Vercel	Web 管理平台框架	SSR/CSR 混合、App Router 文件系统路由
React 19	开	Meta	Web 前端 UI 框架	并发渲染、组件化开发

模块名称	类别	技术来源	承担职责	核心价值
	源			
TypeScript 5.5	开源	Microsoft	前后端类型安全	编译期类型检查，降低联调错误
shadcn/ui + Radix UI	开源	shadcn / Radix	Web 管理台 UI 组件库	无运行时依赖，完全可定制
Tailwind CSS 4	开源	Tailwind Labs	管理台样式体系	实用类优先，快速样式开发
React Native 0.85	开源	Meta	Android 移动端框架	JS 跨平台逻辑 + Kotlin 原生模块
Android SDK / Kotlin	开源	Google	Android 原生模块 (NFC/P2P)	访问 Android 系统 NFC/WiFi P2P API
Three.js + React Three Fiber	开源	Three.js / Poimandres	产品落地页 3D 展示	WebGL 高性能 3D 渲染
libnss-ldap / libpam-ldap	开源	PADL Software	Linux NSS/PAM 与 LDAP 集成	NFS UID 解析指向 LDAP
POSIX ACL (acl 包)	开源	Linux	文件系统访问控制列表	跨协议统一权限执行器

15.2 自研模块清单

模块名称	类别	技术栈	承担职责	核心价值
authd	自研	Go + Gin	统一业务入口：HTTP API、权限管理、文件服务、审计记录、NFC 登录、系统管理	系统核心，跨协议能力协调中枢
LDAP 客户端封装 (ldap/client.go)	自研	Go + go-ldap	UID 分配策略、用户创建/删除、LDAP Bind 封装、NFC 绑定存储	将 LDAP 操作抽象为业务语义接口
权限管理模块 (handler/permission.go +	自研	Go	ACL 读写接口、路径安全校验、共享授权逻辑	POSIX ACL 的业务层封装与安全防护

模块名称	类别	技术栈	承担职责	核心价值
system/files.go)				
审计日志系统 (certified_repo.go)	自研	Go + SQLite	certified_operations 表读写、22 字段完整审计记录	操作留痕，满足合规审计需求
哈希链证明系统 (proof_repo.go)	自研	Go + SHA-256	proof_records 哈希链节点生成与验证、证明包导出	不可篡改的操作证明机制
NFC 认证模块 (handler/nfc.go)	自研	Go (后端) + Kotlin (前端)	NFC 标签绑定与免密登录流程	近场零摩擦登录体验
mDNS 广播服务 (mdns/server.go)	自研	Go + zeroconf	向局域网广播 NAS 服务信息	移动端零配置自动发现
设备 ID 生成 (system/device.go)	自研	Go	基于 /etc/machine-id 生成设备唯一标识	存证溯源的设备标记
Web 管理后台 (nas-web)	自研	Next.js + React + TypeScript	完整 Web 管理界面：仪表盘、文件管理、用户管理、审计日志、存证查询	平台能力的可视化管控入口
Android 移动客户端 (nas-app)	自研	React Native + Kotlin	移动端文件管理、三级连接回退、NFC 登录、WiFi P2P 直连	移动端零配置接入体验
Agent 接入层 (规划中)	自研 (规划)	Python + MCP	自然语言文件检索、权限感知工具调用、RAG 三层检索流水线	NAS 智能化能力的核心扩展
产品落地页 (nas-landing)	自研	Next.js + Three.js	产品特性展示、3D 交互场景	产品市场推广展示面

15.3 开源与自研边界说明

本平台的开源/自研边界遵循以下设计原则：

复用成熟开源组件：协议实现（Samba/NFS/WebDAV/LDAP）、数据库引擎（SQLite/OpenLDAP）、容器运行时（Docker）等领域均采用业界成熟开源方案，不重复造轮子，降低维护风险。

自研业务编排与集成逻辑： 将各开源组件整合为一个协同工作的整体，需要自研的是"如何让这些组件按业务逻辑协作"，包括：跨协议身份同步（LDAP 统一源）、跨协议权限一致性（POSIX ACL 统一执行）、审计覆盖（所有操作经 authd 留痕）、智能能力叠加（Agent + RAG）。

差异化能力自研： 可信存证（哈希链设计）、三级连接回退（mDNS+缓存+P2P）、NFC 碰一碰登录等差异化能力为完全自研，是平台竞争壁垒的核心来源。

16 技术优势与架构特点

16.1 统一身份源架构优势

传统 NAS 产品往往为每个协议维护独立的用户数据库：Samba 有自己的 passdb，NFS 使用本地 `/etc/passwd`，Web 界面使用独立数据库表。用户密码变更需要多处同步，极易出现"Web 密码改了但 SMB 还是旧密码"的问题。

本平台以 OpenLDAP 作为统一身份源，一次变更，所有协议实时生效：

场景	传统 NAS	本平台
用户密码变更	需同步 Samba passdb、Web DB 等多处	修改 LDAP 一处，立即全协议生效
用户账号禁用	需逐一禁用各协议账号	禁用 LDAP 条目，所有协议立即失效
新增协议支持	需重新建立用户同步机制	新协议直接接入 LDAP，无需额外账号管理
审计追溯	各协议日志格式不统一，难以关联	统一用户标识，跨协议操作可追溯到同一身份

16.2 统一权限源架构优势

基于 POSIX ACL 的统一权限源，彻底解决了跨协议权限不一致的历史问题：

- **零漂移：** 权限变更通过内核 ACL 生效，所有协议立即感知，无同步延迟
- **零冗余：** 权限逻辑只有一份（内核 ACL），无需维护多套权限表
- **内核强制：** 即使应用层判断出现 bug，内核层仍会拒绝未授权的文件系统调用

16.3 多协议融合优势

终端类型	最优协议	本平台支持
------	------	-------

终端类型	最优协议	本平台支持
Windows 工作站	SMB	✓ Samba
Linux/Unix 服务器	NFS	✓ NFS
macOS / 跨平台工具	WebDAV	✓ Nginx WebDAV
移动端 / 程序化访问	HTTP API	✓ authd
AI Agent 工具调用	HTTP API + MCP	✓ 规划中

所有协议共享同一套文件系统与权限体系，任何终端访问的都是同一份数据。

16.4 智能扩展能力优势

传统 NAS 是静态文件存储系统，用户必须知道文件路径才能访问。本平台通过 Agent + MCP + RAG 架构，为 NAS 赋予了语义理解能力：

- 用户可用自然语言描述文件查找需求，系统自动定位
- 文件内容可被向量化索引，支持语义相近的模糊检索
- AI 能力不绕过权限体系，安全边界清晰

这一能力在传统 NAS 产品中几乎缺失，是本平台在知识管理与企业应用场景中的核心差异点。

16.5 可信存储能力优势

哈希链可信存证体系为本平台提供了传统 NAS 不具备的合规价值：

- 法律举证能力：操作记录具备不可篡改证明，可作为法律程序的电子证据辅助材料
- 合规审计能力：满足 ISO 27001、等保 2.0 对操作日志完整性与不可篡改性的要求
- 内部风险控制：文件操作全留痕，内部人员的越权访问与数据泄露行为可追溯

16.6 容器化部署优势

- 部署极简：单条命令启动完整服务栈，适合非专业运维人员管理
- 环境一致：开发、测试、生产环境运行完全相同的镜像，消除环境差异
- 升级安全：镜像版本化，升级可回滚，数据卷与容器生命周期解耦
- 资源高效：Go 静态二进制 + 精简 Ubuntu 基础镜像，内存占用远低于 Java 技术栈

16.7 国产化演进路径

本平台在架构设计阶段即预留了国产化演进接口：

国产化方向	当前状态	演进目标
哈希算法	SHA-256	SM3（国密哈希）
签名算法	设备 ID 标记	SM2（国密非对称签名）
硬件安全	软件模拟	PUF 芯片（CCM3302）硬件根信任
加密算法	AES（规划中）	SM4（国密对称加密）

国密算法替换不需要修改上层业务逻辑，仅替换底层算法模块，架构兼容性已提前验证。

17 技术演进路线

17.1 当前已实现能力（V1.0）

能力域	已实现功能	状态
身份认证	OpenLDAP 统一身份源、JWT 认证、LDAP Bind、NFC 绑定与碰一碰登录	生产就绪
权限控制	POSIX ACL 跨协议统一权限、用户私有目录、共享授权	生产就绪
文件服务	HTTP API 文件 CRUD、SMB 共享、NFS 挂载、WebDAV 访问	生产就绪
可信存证	哈希链审计日志（SHA-256）、证明包导出与完整性验证	生产就绪
移动接入	Android App、三级连接回退（mDNS+缓存+P2P）、NFC 登录	生产就绪
设备发现	mDNS Zeroconf 局域网自动广播	生产就绪
管理后台	Web 仪表盘、用户管理、文件管理、审计日志、服务状态	生产就绪
容器化部署	Docker Compose 四容器架构、多阶段构建	生产就绪

能力域	已实现功能	状态
照片自动备份	Android App 登录后自动上传相册照片（Demo 版）	演进中

17.2 近期演进规划（V1.x）

PUF 硬件安全芯片集成

- **当前状态**：代码接口已预留，`signature` 字段存在于 `proof_records` 表，`device_uid` 已作为软件级设备标识写入
- **演进目标**：接入 CCM3302 PUF 芯片，以硬件不可克隆特征生成设备唯一密钥，对哈希链节点进行 SM2 签名
- **技术价值**：将设备溯源从软件级（可伪造）提升至硬件级（不可克隆），存证不可伪造性获得物理根信任保障
- **演进路径**：puf-agent 微服务封装 PUF SDK → authd 调用 puf-agent 签名接口 → 证明包携带 SM2 签名，可用公钥独立验证

SM2/SM3 国密算法替换

- **当前状态**：`hash_algo` 字段已预留算法标识，设计为可插拔
- **演进目标**：将文件哈希算法从 SHA-256 替换为 SM3；将设备签名算法从软件 ID 替换为 SM2 硬件签名
- **技术价值**：满足国内金融、政务、军工等行业对国密算法的合规要求
- **演进路径**：algorithm 模块抽象化 → SM3/SM2 实现注入 → 存量数据以旧算法标识保留，新数据采用新算法

Kerberos SSO 集成

- **当前状态**：架构设计已完成，LDAP 目录结构与 Kerberos Principal 映射已规划
- **演进目标**：在现有 LDAP 身份源基础上叠加 Kerberos 认证协议，支持企业 Active Directory 集成与单点登录（SSO）
- **技术价值**：企业用户无需单独管理 NAS 账号，使用现有 AD 域账号即可登录，与企业 IT 体系深度集成

17.3 中期演进规划（V2.x）

NAS Agent 智能文件管理（完整版）

- **当前状态**：Agent 设计文档已完成（274 行），架构方案确定，Phase 1 开发启动

- **演进目标：**完整实现 Agent + MCP + RAG 三层混合检索的智能文件管理助手，支持自然语言文件查找、内容摘要、跨文件分析
- **实施路线：**
 - Phase 1: 基础工具链（list/search/read/move 等核心工具 + 权限 Hook）
 - Phase 2: RAG 流水线（元数据索引 + FTS5 全文检索）
 - Phase 3: 语义向量检索（bge-m3 向量模型集成）
 - Phase 4: 多模态支持（VLM 图片/PDF 理解）

照片自动备份（正式版）

- **当前状态：**Demo 版已在 Android App 实现，登录后自动上传相册照片至 `/data/<username>/photos/`
- **演进目标：**实现增量备份（仅上传新增照片）、备份进度显示、WiFi 限制（仅 WiFi 时备份）、去重检测
- **技术价值：**家庭用户核心使用场景，降低个人数据对云端的依赖

云边协同

- **演进目标：**支持将本地 NAS 数据选择性同步至云端对象存储（S3/OSS），实现本地优先、云端备份的混合存储架构
- **技术价值：**为有灾备需求的企业用户提供数据冗余保障

17.4 长期演进方向（V3.x）

演进方向	技术目标	战略价值
多节点集群	多台 NAS 设备组成存储集群，统一管理，数据冗余	满足企业级高可用需求
数据加密静态保护	文件存储层透明加密（SM4/AES-256-GCM）	设备被盗时数据不可读
零信任网络访问	WireGuard VPN 集成，支持安全远程访问	突破局域网访问限制
iOS 客户端	扩展移动端覆盖至 iPhone/iPad	扩大用户群体
国产 OS 适配	在统信 UOS、麒麟 OS 上完成认证	政企国产化替代市场
边缘 AI 推理	本地化部署轻量 LLM（Ollama），无需外部 API	数据不出设备的 AI 能力

附录

附录 A：架构决策记录摘要（ADR）

ADR 编号	决策内容	决策日期	核心理由
ADR-001	LDAP 作为唯一身份源	2026-05-15	Samba ldapsam 原生支持，消除多账号同步
ADR-002	POSIX ACL 作为权限真相	2026-05-15	内核强制执行，跨协议零漂移
ADR-003	文件 API 独立于 WebDAV	2026-05-20	HTTP API 需要细粒度权限与审计，WebDAV 无法满足
ADR-004	mDNS 作为局域网发现协议	2026-05-20	零配置，Go 生态成熟库（zeroconf），移动端友好
ADR-005	NFC phone_id 使用 ANDROID_ID	2026-05-28	设备级唯一且稳定，防标签复制攻击
ADR-006	PUF 存证采用哈希链+签名分离	2026-05-25	签名可独立升级（软件→硬件），哈希链不依赖签名即可验证
ADR-007	用户注册不实现事务回滚	2026-05-15	LDAP 与 Linux 账号跨系统，分布式事务成本高，通过幂等清理替代
ADR-010	device_id 统一生成规则	2026-06-08	基于 /etc/machine-id MD5，确保多次重启 ID 稳定
ADR-011	NFC 绑定存储在 LDAP	2026-06-08	与用户身份数据同源管理，简化备份与恢复
ADR-012	container_name 规范化	2026-06-08	跨容器服务依赖通过固定 container_name 引用，避免动态 hostname 问题

附录 B：核心端口清单

端口	协议	服务	说明
8080	HTTP	authd	统一 HTTP API 入口（Web 管理平台与移动 App）

端口	协议	服务	说明
8081	HTTP	Nginx WebDAV	WebDAV 文件访问
445	TCP	Samba	SMB/CIFS Windows 文件共享
2049	TCP/UDP	NFS	网络文件系统
389	TCP	OpenLDAP	LDAP 目录服务
8082	HTTP	nas-agent（规划中）	AI Agent 服务

附录 C：缩略语表

缩略语	全称
NAS	Network Attached Storage（网络附加存储）
LDAP	Lightweight Directory Access Protocol（轻量目录访问协议）
ACL	Access Control List（访问控制列表）
JWT	JSON Web Token
SMB	Server Message Block
NFS	Network File System
WebDAV	Web Distributed Authoring and Versioning
mDNS	Multicast DNS
P2P	Peer-to-Peer
NFC	Near Field Communication
RAG	Retrieval-Augmented Generation（检索增强生成）
MCP	Model Context Protocol
VLM	Vision Language Model（视觉语言模型）
PUF	Physically Unclonable Function（物理不可克隆函数）
SM2/SM3/SM4	国密非对称/哈希/对称算法
SSO	Single Sign-On（单点登录）
VFS	Virtual File System（虚拟文件系统）
POSIX	Portable Operating System Interface
ADR	Architecture Decision Record（架构决策记录）

缩略语	全称
JSB	Java Script Bridge (React Native 原生模块桥接)
FTS	Full-Text Search (全文检索)
ACID	Atomicity, Consistency, Isolation, Durability (数据库事务特性)